ISSUE 33
Define Backtest and Validation Execution Modes #33

June 16, 2026

1 Objective

The purpose of this document is to define how the shared Execution Core operates in two different modes:

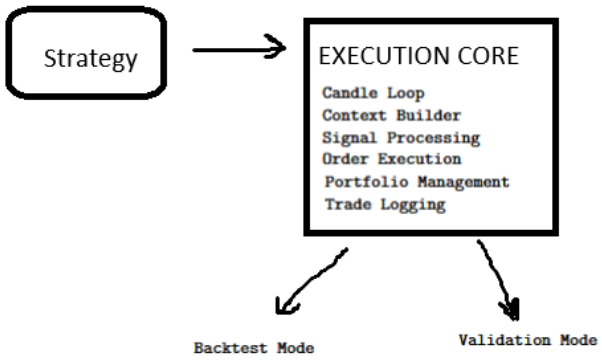
- Historical Backtest Mode
- Fresh-Data Validation Mode

The project uses a single reusable simulation engine for both scenarios.
The execution logic must remain identical in both modes to ensure that strategy performance can be evaluated consistently and reproducibly.
The only differences between modes should be:

- Input data source
- Dataset period
- Triggering workflow
- Result usage

2 Architectural Principle

A key architectural decision of the system is the separation of execution logic from data source selection.



NOTE:
The Execution Core must never know whether it is running a backtest or a validation session.
Its only responsibility is to execute a strategy on a sequence of candles.

3 Backtest Mode Definition

3.1 Purpose

Backtest Mode is used to evaluate a large number of candidate strategies on historical market data.

Its objective is to identify the best-performing strategies according to predefined performance metrics.

3.2 Data Source

The mode consumes historical market data provided by the Data Loader.

Example:

```
BTC/USD
2018-01-01
to
2023-12-31
```

This dataset is considered the training period for strategy evaluation.

3.3 Input

Backtest Mode receives:

```
BacktestRequest

{
strategies: List[Strategy],
candles: List[Candle],
initial_capital: float
}
```

Unlike Validation Mode, Backtest Mode is supposed to execute many strategies sequentially.

3.4 Execution Flow

For each strategy:

1. Initialize portfolio
2. Process all candles
3. Generate trade log
4. Return execution result
5. Calculate performance metrics

3.5 Output

Backtest Mode produces:

- Trade Log
- Final Portfolio State
- Total PL

- Sharpe Ratio
- Max Drawdown
- Win Rate

The Analytics Module uses these results to generate the Top-N strategy ranking.

3.6 Business Purpose

The purpose of Backtest Mode is strategy selection.

Strategies compete against each other using historical data.

Only the highest-ranked strategies proceed to Validation Mode.

4 Validation Mode Definition

4.1 Purpose

Validation Mode performs an additional verification step using data that was not available during the original backtest.

This prevents overfitting and helps determine whether a strategy generalizes to unseen market conditions.

4.2 Data Source

Validation Mode consumes a separate dataset containing newer candles.

Example:

```
BTC/USD
2024-01-01
to
2024-06-30
```

This dataset must never overlap with the backtest dataset.

4.3 Input

Validation Mode receives:

```
ValidationRequest

{
strategy: Strategy,
validation_candles: List[Candle],
initial_capital: float
}
```

Only strategies selected by the ranking system are validated.

4.4 Execution Flow

For each selected strategy:

1. Load validation dataset
2. Initialize portfolio

3. Execute strategy
4. Generate trade log
5. Calculate validation metrics

4.5 Output

Validation Mode produces:

- Validation Trade Log
- Validation Portfolio State
- Validation PL
- Validation Sharpe Ratio
- Validation Max Drawdown
- Validation Win Rate

These metrics are stored separately from backtest results.

4.6 Business Purpose

The purpose of Validation Mode is strategy verification.

The objective is not to discover new strategies but to confirm whether previously selected strategies continue to perform well on unseen data.

5 Differences Between Modes

5.1 Conceptual Difference

Although both modes use identical execution logic, they serve different business objectives.

- Backtest Mode discovers promising strategies.
- Validation Mode confirms strategy robustness.

5.2 Comparison Table

Property	**Backtest Mode**	**Validation Mode**
Purpose	Strategy Selection	Strategy Verification
Data Period	Historical Data	More Recent Data
Dataset Overlap	Allowed only within training period	Must not overlap with backtest data
Strategies Executed	All Candidate Strategies	Top-N Strategies Only
Number of Runs	Potentially Hundreds	Usually Small (Top-N)
Result Usage	Ranking Generation	Ranking Confirmation
Analytics Usage	Primary Selection Metrics	Validation Metrics
Business Goal	Find Winners	Verify Winners

6 Interface-Level Differences

6.1 Shared Interface

Both modes ultimately invoke the same Execution Core interface:

```
ExecutionEngine.run(
strategy,
candles,
initial_capital
)
```

This guarantees identical execution behavior.

6.2 Backtest Wrapper

Backtest Mode acts as an orchestrator:

```
for strategy in strategies:

```
execution_engine.run(
    strategy,
    historical_candles,
    capital
)
```
```

6.3 Validation Wrapper

Validation Mode acts as a validator:

```
for strategy in top_n_strategies:

```
execution_engine.run(
    strategy,
    validation_candles,
    capital
)
```
```

The execution engine itself remains unchanged.

7 Required Outputs

The following outputs are mandatory for both modes.

7.1 Execution Core Outputs

- Trade Log
- Portfolio State

7.2 Analytics Outputs

- Total PL
- Sharpe Ratio
- Maximum Drawdown
- Win Rate

7.3 Database Outputs

Results must be persisted in PostgreSQL.

Required tables include:

- trades
- metrics
- strategy$_r ankings$
- validation$_r esults$

8 Documentation Update

The project documentation must be updated with the following sections:

1. Execution Modes Overview
2. Backtest Mode Specification
3. Validation Mode Specification
4. Interface Contracts
5. Execution Flow Diagrams
6. Mode Comparison Table

9 Architectural Rule

Backtest Mode and Validation Mode must never implement their own simulation logic.

All candle-by-candle processing, signal handling, portfolio updates, and trade logging must be delegated to the shared Execution Core.

This guarantees:

- Deterministic execution
- Consistent strategy evaluation
- Reduced code duplication
- Easier maintenance
- Simplified testing

10 Conclusion

The system defines two operational modes:

- Historical Backtest Mode
- Fresh-Data Validation Mode

Both modes share the same Execution Core and differ only in data source, dataset period, and business objective.

This architecture ensures that strategy evaluation remains consistent while supporting both strategy discovery and strategy validation workflows.